

Personal Loan Ledger Mobile Application

Software Design Specification (SDS)

Shared Loan Tracking, Reminders, Repayment Confirmation, Dashboard Analytics, and Bank Profile Sharing

Document	Software Design Specification (SDS)
Version	Version 1.0 - Product Foundation
Date	September 2026
Purpose	Translates the product requirements into implementable modules, APIs, state transitions, security controls, and engineering standards.

Product scope note: The application tracks peer-to-peer loans and repayment acknowledgements. It does not custody funds, execute bank transfers, provide credit scoring, or automatically enforce legal debt collection in the initial release.

Table of Contents

1. Design Goals and Technical Scope
2. Technology Stack and Engineering Standards
3. React Native Application Structure
4. Backend Application Structure
5. API Design Conventions
6. Authentication and Session Design
7. Authorization Model
8. Friendship Module Design
9. Loan Module Design
10. Terms Versioning and State Machine
11. Repayment Module Design
12. Bank Profile Security Design
13. Dashboard Query and Aggregation Design
14. Notification and Scheduling Design
15. Audit/Event Timeline Design
16. File and Media Handling
17. Validation and Error Model
18. Concurrency, Idempotency, and Transactions
19. Caching and Performance Strategy
20. Offline and Synchronization Strategy
21. Security Engineering Controls
22. Testing Strategy
23. CI/CD and Environment Strategy
24. Observability and Operations
25. Deployment and Scaling
26. Failure Recovery and Data Repair
27. API Endpoint Catalog
28. Implementation Phases and Definition of Done

1. Design Goals and Technical Scope

The SDS specifies how the system will implement the SRS. It intentionally favors a modular monolith for the first release: simpler than microservices while preserving clear module boundaries for future extraction.

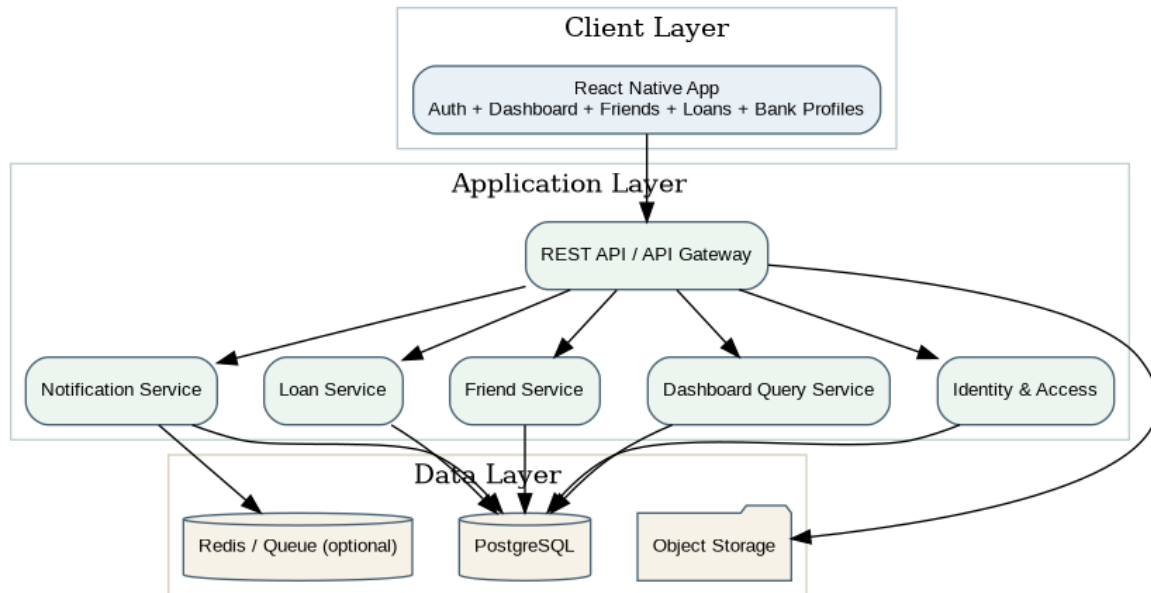


Figure 1. Recommended logical architecture for the first production release.

- React Native client is a consumer of a versioned backend API and never acts as the source of truth for financial state.
- Backend modules encapsulate identity, friendships, loans, repayments, bank profiles, notifications, and dashboard queries.
- PostgreSQL is the transactional source of truth.
- Background jobs handle reminders and retryable notification delivery.
- A modular-monolith deployment minimizes distributed-system risk while allowing horizontal scaling.

2. Technology Stack and Engineering Standards

Layer	Recommended Choice	Reason
Mobile	React Native + TypeScript	Cross-platform client with shared domain types and mature ecosystem.
Navigation	React Navigation or Expo Router	Structured authenticated/public flows and deep links.
Server state	TanStack Query	Caching, retries, invalidation, background refresh.
Client forms	React Hook Form + schema validation	Predictable forms and field-level validation.
Backend	Node.js + NestJS + TypeScript	Strong modular structure, DI, guards, validation, OpenAPI support.
Database	PostgreSQL	Transactions, constraints, decimal/numeric types, JSON metadata, robust indexing.
ORM	Prisma or TypeORM	Migrations, typed access, transaction support; team should standardize on one.
Jobs	BullMQ + Redis or managed queue	Delayed reminders, retries, dead-letter handling.
Push	FCM/APNs via a provider abstraction	Cross-platform push with delivery diagnostics.
Storage	S3-compatible object storage	Avatars and future proof/receipt attachments.
API docs	OpenAPI/Swagger	Contract visibility and client generation opportunities.

- TypeScript strict mode enabled across client and backend.
- ESLint/formatter and pre-commit/CI checks standardized.
- Secrets and environment-specific configuration injected at deployment, never committed.

3. React Native Application Structure

Folder / Module	Responsibility
src/app	Providers, navigation root, error boundary, app bootstrap.
src/features/auth	Sign-in, registration, verification, recovery.
src/features/profile	Profile and preferences.
src/features/friends	Discovery, requests, connections, blocking.
src/features/loans	Request, terms, details, timeline, status views.
src/features/repayments	Submit and confirm repayment.
src/features/bankProfiles	Manage, reveal, share, revoke.
src/features/dashboard	Metric cards, due lists, action queues.
src/features/notifications	In-app notification inbox and deep links.
src/services/api	Typed HTTP client, auth interceptors, error mapping.
src/components	Reusable UI primitives.
src/domain	Money, dates, status enums, pure calculation helpers.

- Feature folders own screens, hooks, validation schemas, query keys, and feature-specific components.
- Domain calculations are pure functions with unit tests.
- Navigation params use stable IDs; sensitive values should not be placed in deep-link URLs.

4. Backend Application Structure

Backend Module	Key Services
AuthModule	Registration, verification, login, refresh/session revocation.
UsersModule	Profile, preferences, deletion workflow.
FriendshipsModule	Search/invite, request lifecycle, block/remove.
LoansModule	Loan request, term proposal, acceptance, state transitions, detail queries.
RepaymentsModule	Submit, confirm, reject, balance application.
BankProfilesModule	Encrypt/decrypt, preferred profile, share authorization, revoke.
DashboardModule	Currency-grouped metrics and due/pending summaries.
NotificationsModule	In-app notification creation, push abstraction, preferences.
JobsModule	Reminder scheduling, overdue transitions, retries.
AuditModule	Append-only event creation and timeline projection.
AdminModule	Support-only resolution paths with strong audit controls.

5. API Design Conventions

- Base path: /api/v1.
- JSON request/response payloads with ISO-8601 UTC timestamps and ISO currency codes.
- Pagination uses cursor-based pagination for histories and notifications where scale may grow.
- All state-changing endpoints support a client-generated idempotency key where duplicate retries are plausible.
- Use consistent envelope or direct-resource convention; avoid mixing styles.
- Expose machine-readable error code, human message, and safe field errors.

HTTP	Usage
200	Successful read/update.
201	Resource created.
202	Asynchronous work accepted if used.
204	Successful action with no body.
400	Malformed request or invalid state-independent input.
401	Not authenticated.
403	Authenticated but not authorized.
404	Resource not found or intentionally hidden.
409	State conflict / duplicate / invalid transition.
422	Semantically invalid fields.
429	Rate limited.

6. Authentication and Session Design

- Access tokens are short-lived; refresh/session credentials are stored using platform secure storage.
- Server stores refresh token hashes or session records so individual devices can be revoked.
- Session records include user ID, device label, created/last-used timestamps, revocation timestamp, and optional risk metadata.
- Verification codes are hashed, short-lived, attempt-limited, and rate-limited.
- Password hashing uses Argon2id or bcrypt with current cost guidance.
- Sensitive bank reveal may require recent authentication depending on risk tolerance.

7. Authorization Model

Resource	Authorization Rule
User profile	Self may edit; connected users may see only allowed public fields.
Friendship	Only requester/addressee may access; block status changes visibility.
Loan	Only borrower, lender, and narrowly scoped support roles may access.
Repayment	Only parties to parent loan; submitter and confirmer roles validated by state.
Bank profile	Owner has full access; recipient only through active share scope.
Notification	Only owning user.
Audit timeline	Borrower/lender may see user-safe event projection; admin sees additional safe metadata if authorized.

Implementation rule: Authorization is enforced in service/repository logic or guards using database relationships. Never trust borrower_id, lender_id, or owner_id directly from the client.

8. Friendship Module Design

- Canonical pair uniqueness can be enforced by storing `user_low_id` and `user_high_id` alongside directional requester/addressee IDs.
- Friend request acceptance runs in a transaction and rejects blocked relationships.
- Removing a friendship changes connection status but does not cascade-delete loans.
- A block creates a durable relationship barrier checked before new loan requests, invitations, or bank sharing.
- Friend list queries include derived per-currency balances through optimized aggregate queries or cached projections.

9. Loan Module Design

- Loans service is the only module allowed to change loan.status directly.
- Loan creation validates friendship, block status, and borrower/lender identity.
- Acceptance writes accepted_at, accepted_by, a loan_terms version, and a loan event in one transaction.
- Current outstanding balance is either derived from confirmed repayments or maintained as a carefully updated denormalized field with reconciliation checks.
- Original principal/currency never change after activation; amendments create new term versions and events.

Core Method	Behavior
requestLoan()	Create REQUESTED loan and event.
proposeTerms()	Create draft/versioned proposed terms; does not activate loan.
acceptLoan()	Validate state, persist accepted terms, activate, schedule reminders.
markOverdue()	Background transition when due time passes and balance > 0.
cancelPendingLoan()	Allow borrower cancellation before acceptance.
getLoanDetail()	Return role-aware detail, terms, balances, timeline, bank share availability.

10. Terms Versioning and State Machine

Transition	Guard	Atomic Side Effects
REQUESTED -> ACTIVE	Caller is lender; valid terms; not blocked; request unresolved.	Terms snapshot, accepted timestamp, audit event, reminder jobs.
ACTIVE -> OVERDUE	Server time > due_at and outstanding > 0.	Status change event, overdue notifications.
ACTIVE/OVERDUE -> REPAYMENT_PENDING	Caller is borrower; valid amount; no conflicting pending claim.	Repayment row, event, lender notification.
REPAYMENT_PENDING -> COMPLETED	Caller is lender; confirmed amount closes balance.	Repayment confirm, loan close, event, borrower notification.
REPAYMENT_PENDING -> ACTIVE/OVERDUE	Caller is lender; rejection.	Repayment reject, status calculated from due date, event, borrower notification.

- Use optimistic concurrency via row version/update timestamp or SELECT ... FOR UPDATE for critical transitions.
- State checks and writes must occur inside one database transaction.
- Idempotency keys protect against duplicate mobile retries.

11. Repayment Module Design

- Repayment submission owns an independent status: PENDING, CONFIRMED, REJECTED, CANCELLED if policy allows.
- The service calculates maximum allowable claim from current outstanding balance; client-submitted totals are advisory only.
- A confirmed repayment creates ledger effects and can close the loan.
- Partial repayment support can be enabled later without schema migration if amount and confirmed_amount are already represented.
- Proof attachments reference object-storage IDs rather than embedding file data in the relational database.

12. Bank Profile Security Design

- Store account identifier encrypted in an *_enc byte/string column and optionally store a non-sensitive last4/masked_display field for list screens.
- Encryption/decryption runs in a dedicated service backed by KMS or rotated application keys.
- Do not expose encrypted ciphertext to client.
- Bank shares reference profile, owner, recipient, optional loan, created_at, revoked_at, and access purpose.
- API response returns full identifier only to owner or active authorized recipient, ideally through a specific reveal endpoint that can be audited.
- Push notifications say “Payment details shared” rather than including account numbers.

13. Dashboard Query and Aggregation Design

- Dashboard endpoint returns currency-grouped objects rather than one ambiguous total.
- Use SQL conditional aggregates across loans and confirmed repayments; filter by user role and status.
- Recommended endpoint returns summary plus action queues (pending requests, repayment confirmations, due soon).
- For early scale, compute on request with well-designed indexes. At larger scale, introduce materialized views or projection tables updated transactionally/eventually with reconciliation jobs.

Derived Field	Calculation
receivableOutstanding	SUM(outstanding_balance) where lender_id=user and status open, grouped currency.
payableOutstanding	SUM(outstanding_balance) where borrower_id=user and status open, grouped currency.
netPosition	receivableOutstanding - payableOutstanding per same currency.
overdueReceivable	SUM(outstanding) where lender=user and due_at<now and balance>0.
dueSoon	Open loans with due_at in [now, now+window].
pendingActions	Incoming loan requests + repayment confirmations + optional term acknowledgements.

14. Notification and Scheduling Design

- A scheduler enqueues reminder jobs at acceptance time based on due_at and user reminder policy.
- Job keys are deterministic, e.g., loan:{id}:due-3d:{user_id}, so duplicate scheduling can be ignored.
- If due date changes through a valid amendment, obsolete jobs are cancelled/invalidated and new ones scheduled.
- A periodic reconciliation job finds open loans approaching/past due and ensures expected reminder jobs exist.
- Push delivery is abstracted behind NotificationProvider; provider errors are categorized as transient/permanent.
- In-app notification row is created even if push token is absent.

15. Audit/Event Timeline Design

- Audit events are append-only and created in the same transaction as the state change they describe.
- Use event_type enum + metadata_json for event-specific safe details while preserving typed foreign keys for primary entities.
- Generate a user-facing timeline projection that maps internal events to plain language.
- Admin actions include admin_actor_id/reason but are never represented as if performed by borrower/lender.
- Event deletion requires exceptional data-retention process, not ordinary CRUD.

16. File and Media Handling

- Profile photos and future proof-of-payment attachments use pre-signed upload URLs or backend-mediated uploads.
- Validate MIME type, file size, image dimensions, and optional malware scanning before marking upload usable.
- Object keys use random IDs, not personal names or account numbers.
- Access uses private buckets and time-limited signed download URLs.
- Deleting a file marks association removed and schedules storage cleanup according to retention rules.

17. Validation and Error Model

Error Code	Example Meaning
FRIENDSHIP_REQUIRED	Users are not connected for a new loan request.
LOAN_INVALID_STATE	Requested action is not valid for current loan status.
LOAN_TERMS_INVALID	Principal, rate, currency, or due date failed validation.
REPAYMENT_EXCEEDS_BALANCE	Claimed repayment is greater than outstanding balance.
BANK_SHARE_NOT_AUTHORIZED	Caller lacks an active share for the bank profile.
IDEMPOTENCY_CONFLICT	Same key reused with different payload.
RATE_LIMITED	Too many sensitive attempts.
SESSION_REAUTH_REQUIRED	Recent authentication required for reveal/security action.

- Validation schemas are shared conceptually but server validation remains authoritative.
- Client maps safe field errors to form controls and generic server errors to retry/support UI.

18. Concurrency, Idempotency, and Transactions

- Every financial transition uses a database transaction.
- Use unique constraints to prevent duplicate active idempotency records and duplicate accepted friendships.
- Critical status update queries include expected prior status in WHERE clause or row locking.
- Idempotency table stores user, key, route/action, request hash, response snapshot/status, created_at, and expiry.
- Notification scheduling is idempotent with unique job keys.
- Reconciliation jobs periodically compare denormalized balances/dashboard projections against source repayments.

19. Caching and Performance Strategy

- Do not cache full bank profile responses containing decrypted sensitive values.
- Cache public/configuration data and possibly dashboard summaries for short TTL only after correctness is proven.
- Use database indexes first: (lender_id,status,currency_code), (borrower_id,status,currency_code), (due_at,status), friendship canonical pair, notification user/read, repayment loan/status.
- Avoid N+1 query patterns in friend and loan lists.
- Use keyset/cursor pagination for large history tables.

20. Offline and Synchronization Strategy

- MVP is online-first for state-changing financial actions.
- Client may cache last successful dashboard and loan list for read-only offline display with a visible “last updated” timestamp.
- Creating/accepting loans, sharing bank details, and confirming repayment require online server confirmation.
- When reconnecting, invalidate relevant query keys and refetch authoritative state.
- Do not queue sensitive financial mutations indefinitely on-device without explicit UX and idempotency semantics.

21. Security Engineering Controls

Control	Design
OWASP API authorization	Per-object relationship checks and deny-by-default guards.
Mass assignment	DTO allowlists; server sets owner/actor fields.
Injection	Parameterized ORM queries and validated sort/filter fields.
Sensitive data	Field-level encryption, masking, log redaction, secure storage.
Abuse/rate limit	IP + account limits for auth, discovery, reveal, request spam.
CSRF	Not applicable to bearer-token native API in same way as cookie web app; if web admin uses cookies, apply CSRF protection.
Replay	Idempotency keys, token expiry, recent-auth requirement for sensitive actions.
Dependencies	Automated vulnerability scanning and lockfile review.
Mobile secrets	No long-term private keys/API secrets in app bundle; public provider IDs only where unavoidable.

22. Testing Strategy

- Unit tests: money calculations, status guards, date/reminder calculations, masking/encryption wrappers.
- Service tests: friendship rules, loan acceptance, repayment confirmation, authorization, bank shares.
- Database/integration tests: transactions, unique constraints, concurrency, migrations.
- API contract tests: endpoint schemas and error codes.
- Mobile component tests: forms, role-specific rendering, dashboard cards.
- End-to-end tests: register -> connect -> request -> accept -> due reminder simulation -> repay -> confirm.
- Security tests: IDOR attempts, blocked friendship, reveal authorization, rate limits.
- Load tests: dashboard aggregation and loan list endpoints at target scale.

23. CI/CD and Environment Strategy

- Environments: local, test/CI, staging, production; no production credentials in non-production.
- Migration files are immutable after release and applied through deployment pipeline with backup/rollback plan.
- CI runs lint, typecheck, unit tests, integration tests, dependency/security scans, and build checks.
- Mobile releases use signed build pipeline (e.g., EAS or native CI) with environment-specific API configuration.
- Backend uses container images pinned by digest/version and progressive deployment where available.

24. Observability and Operations

Signal	Example Alert / Dashboard
API errors	5xx rate and endpoint-specific spikes.
Latency	p50/p95/p99 by endpoint.
Jobs	Reminder queue depth, oldest job age, failed/retried jobs.
Push	Provider rejection rate, invalid token rate.
Database	CPU, storage, connections, slow queries, replication/backup health.
Business integrity	Failed state transitions, balance reconciliation mismatch, duplicate idempotency conflicts.
Security	Auth brute-force blocks, suspicious reveal rate, blocked-user request attempts.

25. Deployment and Scaling

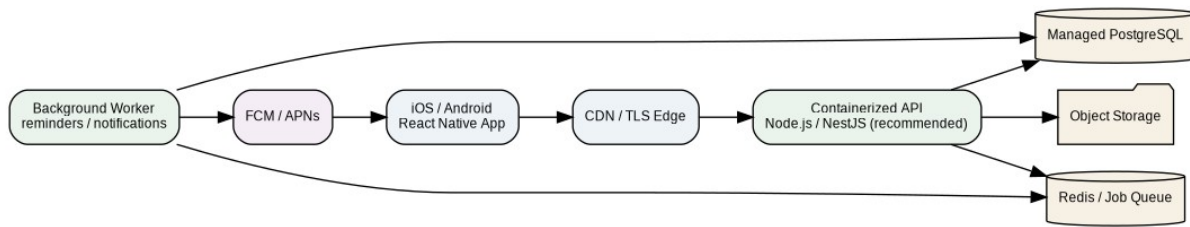


Figure 2. Reference deployment topology with separate API and background worker processes.

- API instances are stateless and horizontally scalable behind a TLS edge/load balancer.
- Worker instances consume durable delayed jobs; at-least-once delivery is handled with idempotent processors.
- Managed PostgreSQL provides automated backups, point-in-time recovery if available, and encrypted storage.
- Redis/job queue is not authoritative for loans; lost queue data can be reconstructed by reconciliation jobs.
- Static object storage is private with lifecycle rules.

26. Failure Recovery and Data Repair

- Database restores must be tested, not assumed.
- A balance reconciliation command compares loans to confirmed repayments and reports mismatches without silently fixing them.
- Administrative repair uses explicit commands with dry-run, reason, actor, and audit event.
- If notification queue is lost, reminder reconciliation reconstructs future reminders from loan due dates.
- If a push provider is unavailable, in-app notifications and loan state continue to function.
- Runbooks should cover DB incident, queue failure, push outage, secret rotation, compromised account, and accidental release rollback.

27. API Endpoint Catalog

Endpoint	Purpose
POST /auth/register	Register account
POST /auth/verify	Verify email/phone
POST /auth/login	Create session
POST /auth/refresh	Refresh session
GET /me	Current profile
PATCH /me	Update profile/preferences
GET /friends	List accepted connections
POST /friend-requests	Send request
POST /friend-requests/{id}/accept	Accept friendship
POST /loans	Create borrower loan request
GET /loans	List/filter loans
GET /loans/{id}	Loan detail
POST /loans/{id}/terms	Lender proposes/sets terms
POST /loans/{id}/accept	Accept and activate loan
POST /loans/{id}/reject	Reject request
POST /loans/{id}/repayments	Borrower submits repayment
POST /repayments/{id}/confirm	Lender confirms receipt
POST /repayments/{id}/reject	Lender rejects claim
GET /bank-profiles	Owner bank profiles
POST /bank-profiles	Create bank profile
POST /bank-profiles/{id}/share	Share with counterparty/loan
POST /bank-profile-shares/{id}/revoke	Revoke share
GET /loans/{id}/payment-profile	Retrieve shared masked/revealed profile as authorized
GET /dashboard	Currency-grouped metrics
GET /notifications	In-app notification list
POST /notifications/{id}/read	Mark read

28. Implementation Phases and Definition of Done

Phase	Definition of Done
Phase 1 Foundation	Repo, CI, auth, profile, database migrations, environments, API conventions.
Phase 2 Social	Friend discovery/invite, requests, accepted list, block/remove.
Phase 3 Loans	Request, terms, acceptance, details, history, audit events.
Phase 4 Dashboard	Currency-grouped aggregates, filtered drill-down lists.
Phase 5 Bank Profiles	Encrypted storage, preferred profile, share/revoke/reveal.
Phase 6 Repayment	Submit, confirm/reject, close loan, history.
Phase 7 Notifications	Push/in-app inbox, scheduling, overdue jobs, retries.
Phase 8 Hardening	Security testing, concurrency/idempotency, performance, monitoring, backups, app-store readiness.